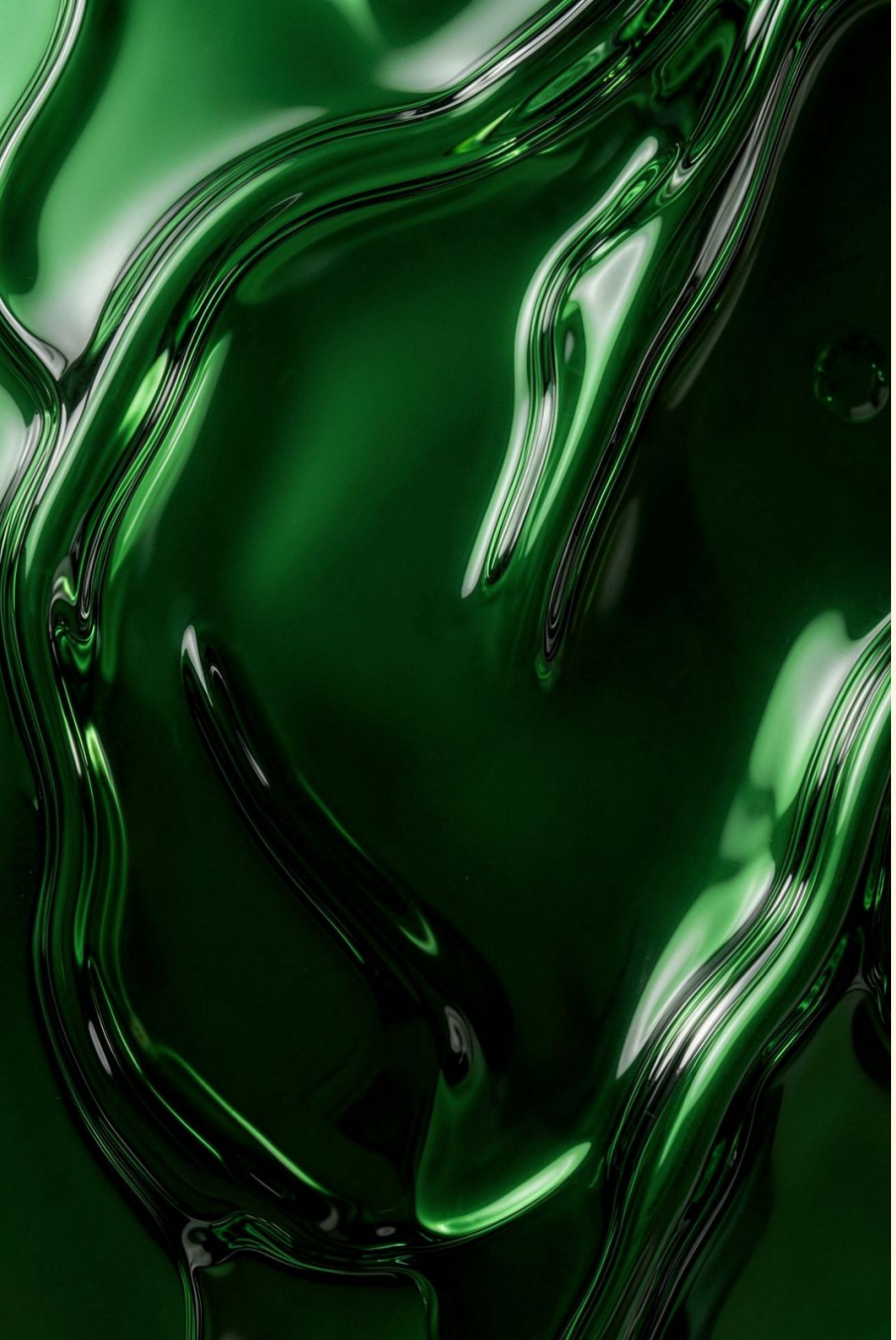


CSS Gradients, Transformations, Transitions, Animations, Filters & Tooltips

A practical guide to modern, interactive web styling — from smooth backgrounds to animated components, using HTML & CSS only.

What We'll Cover



Gradients

Smooth color transitions as backgrounds

Transitions

Animate property changes smoothly



Filters

Visual effects on images and elements



Transformations

Move, rotate, scale, and skew elements



Animations

Multi-stage sequences with @keyframes



Tooltips

Informational popups using pure CSS

CSS Gradients: Three Types

Linear

Colors flow along a straight line

```
linear-gradient(to right, blue, purple)
```

Radial

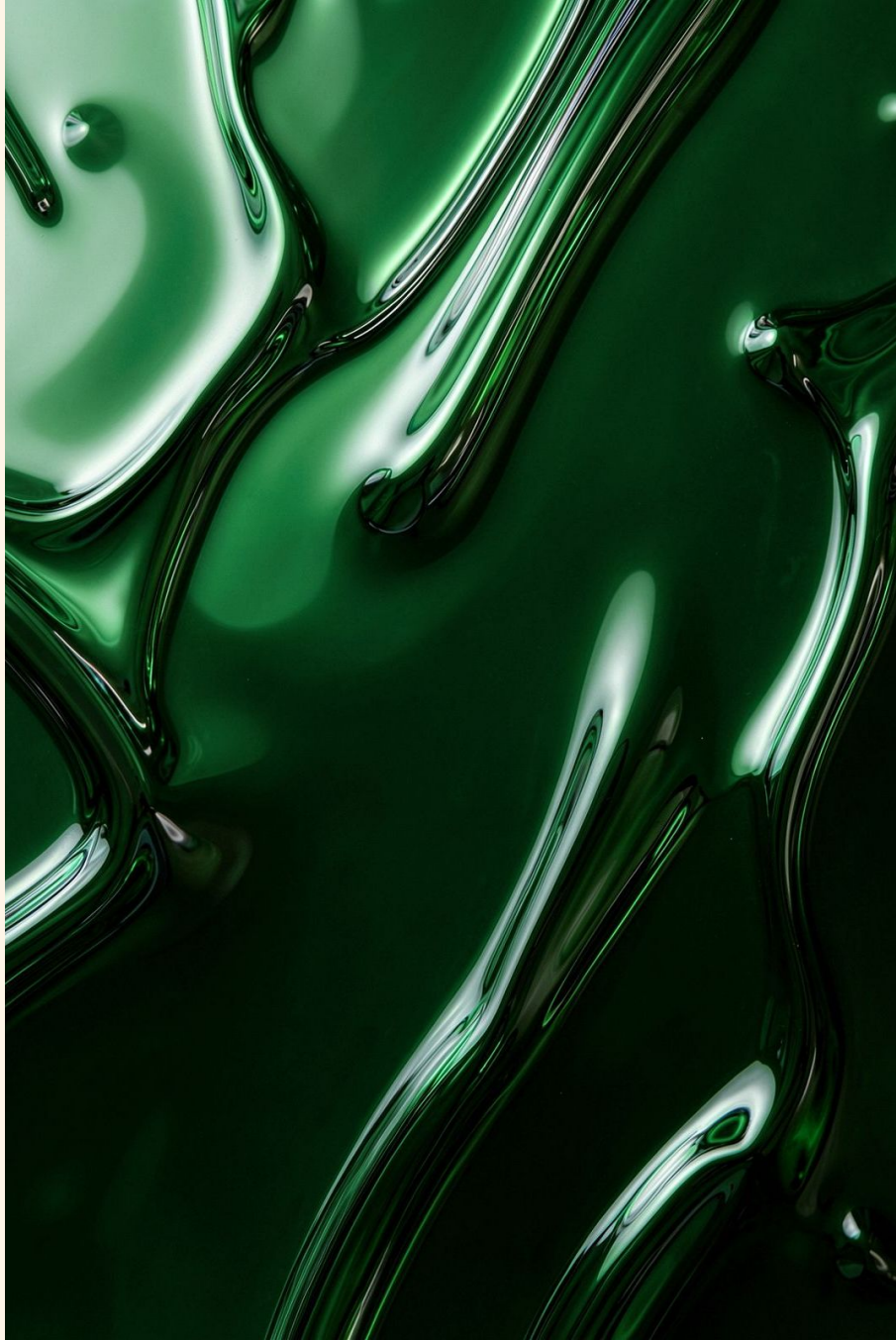
Spreads outward from a center point

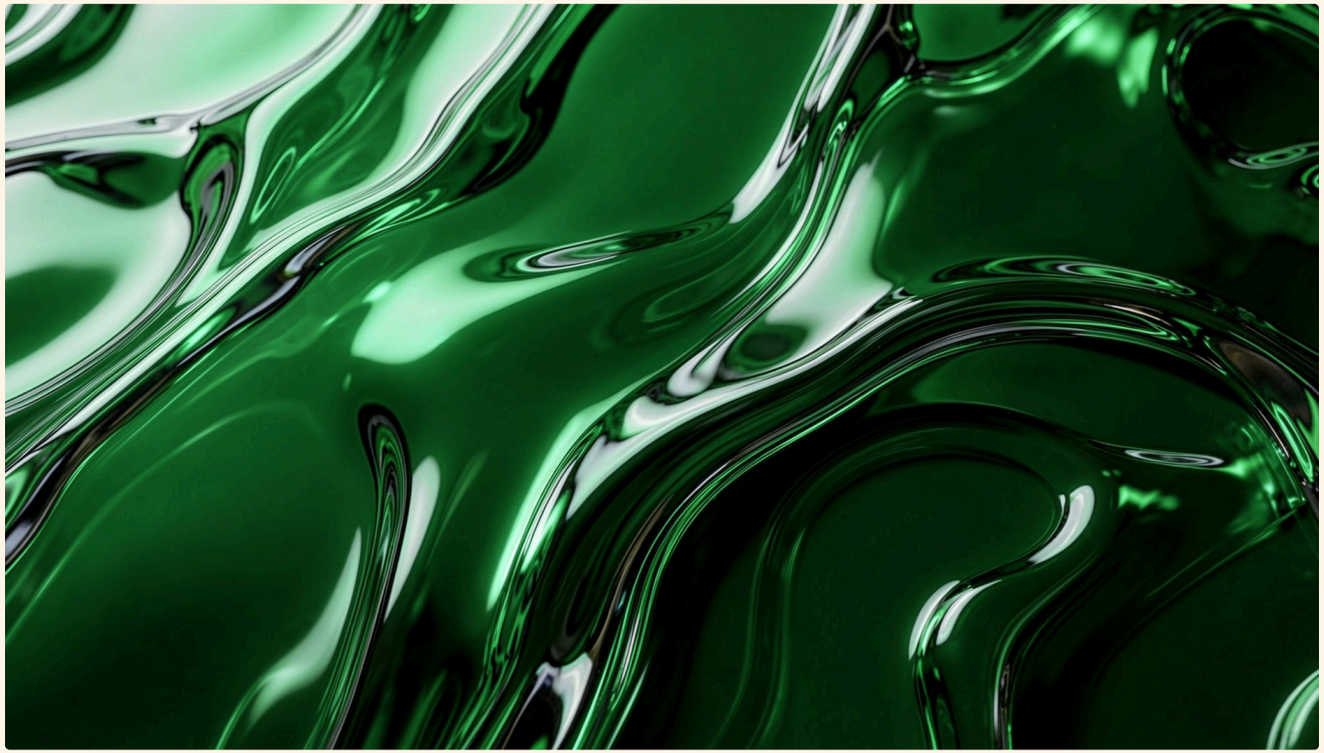
```
radial-gradient(circle, white, blue)
```

Conic

Rotates colors around a center

```
conic-gradient(red, yellow, green, blue, red)
```





Gradient Color Stops & Common Uses

Control exact positions with percentage stops to create precise, multi-color transitions:

```
linear-gradient(to right,
  #ff0000 0%,
  #ffff00 50%,
  #00ff00 100%)
```

Common Uses

- Hero sections & headers
- Buttons & cards
- Promotional banners
- Decorative backgrounds

CSS Transformations

translate()

Moves an element along X and Y axes

```
transform: translate(50px, 20px)
```

rotate()

Rotates an element by degrees

```
transform: rotate(45deg)
```

scale()

Resizes — 1.5 = 150%, 0.5 = 50%

```
transform: scale(1.5)
```

skew()

Tilts an element along an axis

```
transform: skew(20deg)
```

Combine multiple transforms on one line: `transform: translateX(50px) rotate(20deg) scale(1.2)`

Transform Origin

`transform-origin` sets the **pivot point** for any transformation — think of it as the pin that holds the element in place while it moves.

- **top left** — rotates from the top-left corner
- **center** — rotates from the middle (default)
- **bottom right** — rotates from the bottom-right corner

Essential for rotating cards, doors, menus, and interactive components.

Code Example

```
.box {
  transform-origin: top left;
  transform: rotate(45deg);
}
```

Change `transform-origin` to see how the pivot point shifts the visual result of the same rotation.

CSS Transitions: Smooth Property Changes

❌ Without Transition

Property changes **instantly** on hover — jarring and abrupt.

```
button:hover {
  background-color: black;
}
```

✅ With Transition

Change happens **smoothly** over a set duration — polished and professional.

```
button {
  transition: background-color
    0.3s ease;
}
button:hover {
  background-color: black;
}
```

One line of CSS — `transition` — is all it takes to go from clunky to smooth.

Transition Properties

1. ## transition-property

   Which CSS property to animate (e.g. `color`, `transform`, `all`)

2. ## transition-duration

   How long the transition takes — e.g. `0.5s`

3. ## transition-timing-function

   Speed curve: `ease`, `linear`, `ease-in`, `ease-out`, `ease-in-out`

4. ## transition-delay

   Wait before starting — e.g. `0.2s`. Shorthand: `transition: all 0.3s ease;`



Transformations + Transitions Together

Combining both creates smooth, interactive hover effects that feel natural and polished.

Card Lift Effect

```
.card {
  transition: transform 0.3s ease;
}
.card:hover {
  transform: translateY(-10px);
}
```

Button Grow Effect

```
.button {
  transition: transform 0.2s ease;
}
.button:hover {
  transform: scale(1.05);
}
```

The `transition` property lives on the **base element**, not the `:hover` state — this ensures the animation plays in both directions.

CSS Animations with @keyframes

Animations run through **multiple stages** over time — **no user interaction required**. Define stages with `@keyframes`, then apply with the `animation` property.

- Declare stages at 0%, 50%, 100% (or use `from` / `to`)
- Apply to any element with the animation shorthand
- Use `infinite` for looping effects

Example: moveBox Animation

```
@keyframes moveBox {
 0% { transform: translateX(0); }
 50% { transform: translateX(200px); }
 100% { transform: translateX(0); }
}

.box {
 animation: moveBox 3s
 ease-in-out infinite;
}
```

Animation Properties

01

animation-name

Which `@keyframes` block to use

02

animation-duration

How long one full cycle takes (e.g. `2s`)

03

animation-iteration-count

Number of repeats — use `infinite` for looping

04

animation-direction

`alternate` reverses on each cycle for a back-and-forth effect

05

animation-delay / timing-function

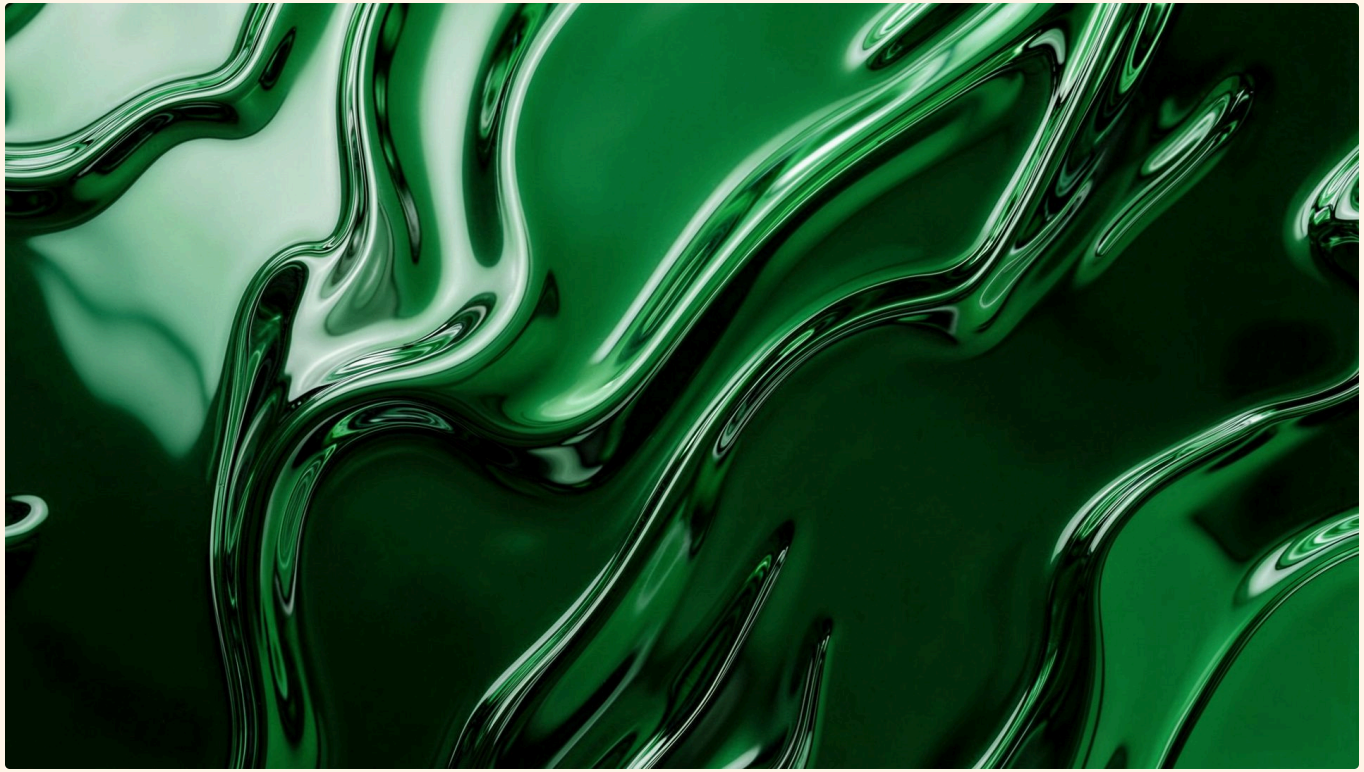
Delay the start and control the speed curve

Animation vs. Transition

Knowing when to use each tool is key to writing clean, intentional CSS.

	Transition	Animation
Trigger	State change (`:hover`, `:focus`)	Runs automatically
Defined with	CSS property values	`@keyframes`
Best for	Simple interactions	Complex sequences
States	Two (start → end)	Multiple stages
Example	Button hover	Loading spinner

CSS Filters



Apply visual effects to images and elements using the `filter` property. Filters can be stacked for layered effects.

```
filter: grayscale(100%)
        brightness(80%)
        contrast(120%);
```

blur(5px)

Softens the element

grayscale(100%)

Removes all color

brightness(50%) / contrast(150%)

Darkens or sharpens

sepia / saturate / hue-rotate

Color-shifting effects

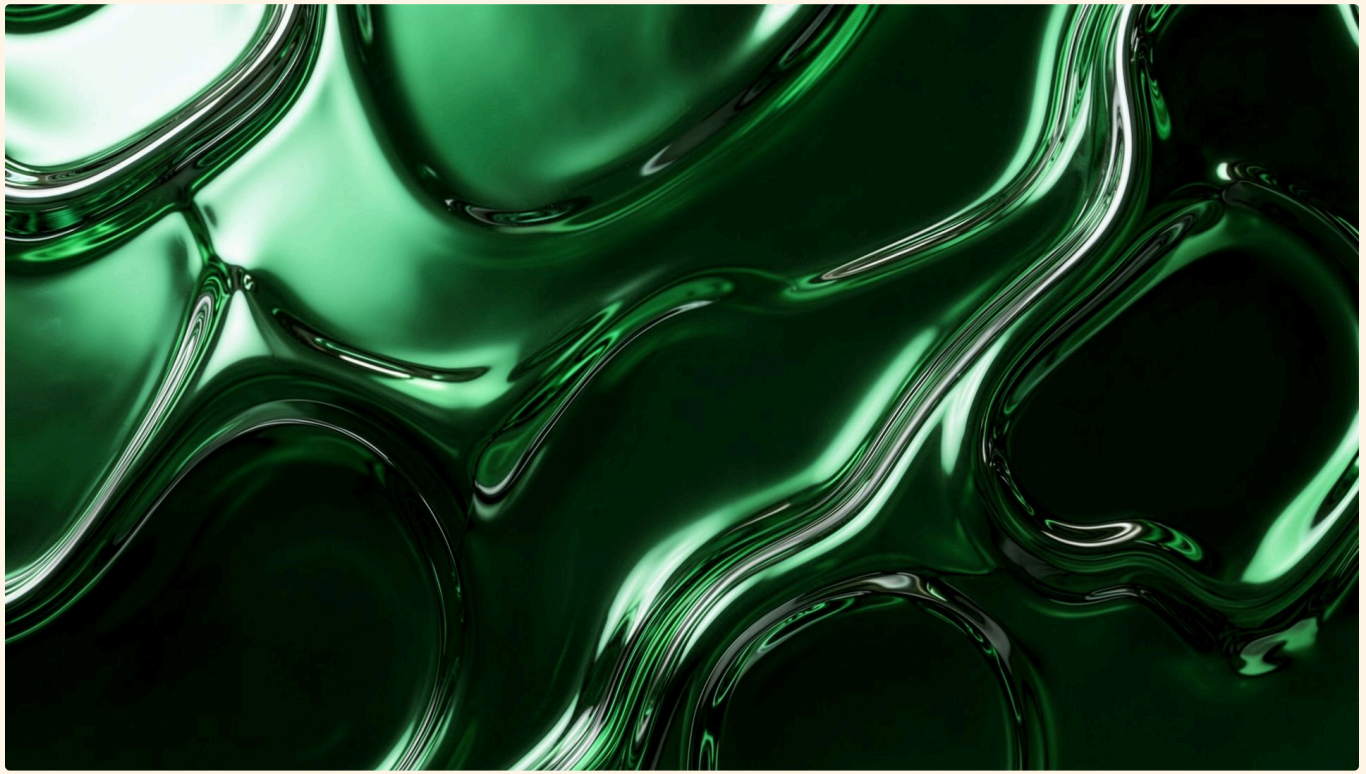
Filters + Transitions: A Classic Effect

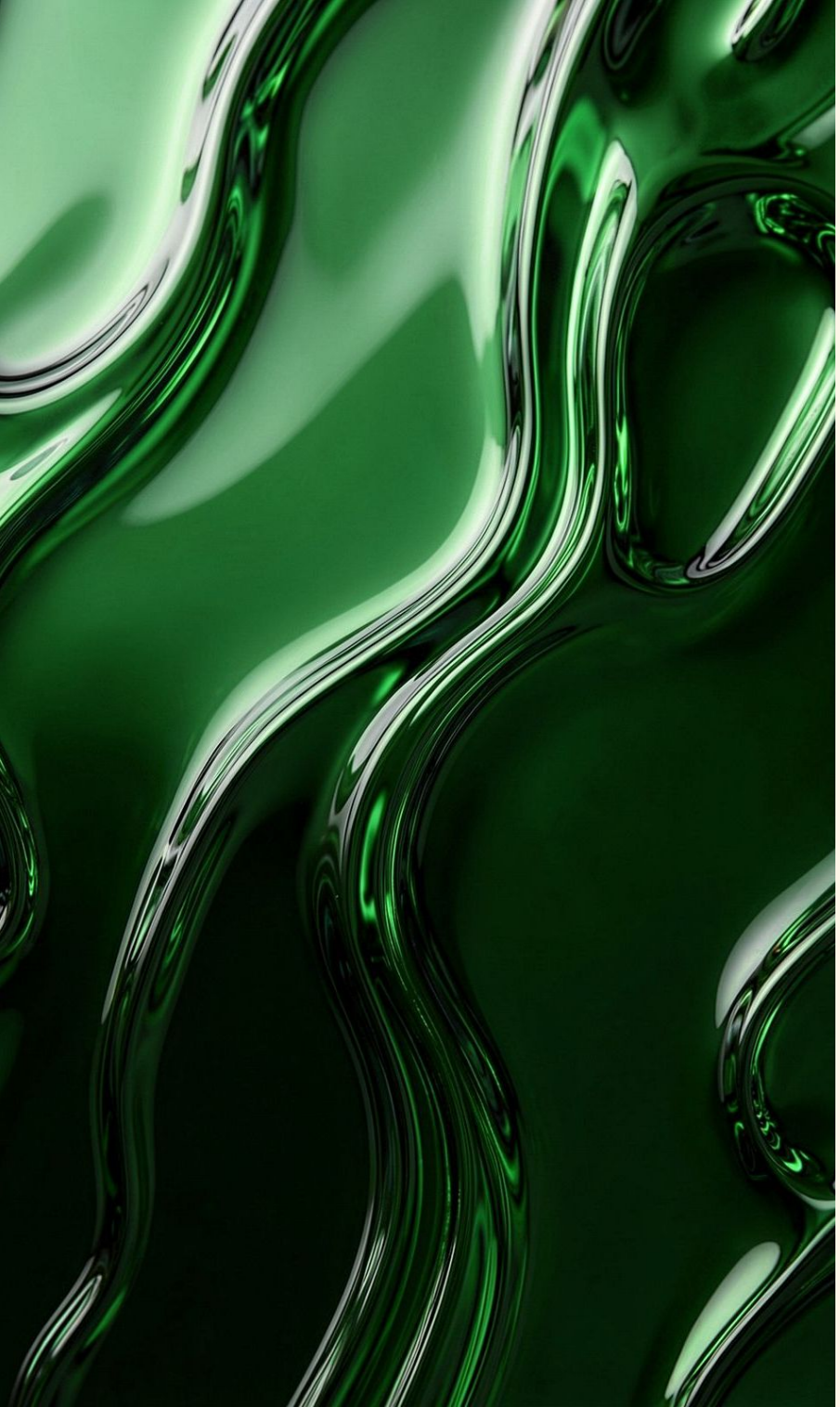
The Grayscale Reveal

Start an image in grayscale — reveal full color on hover. One of the most elegant effects in CSS, using only two properties.

```
img {
  filter: grayscale(100%);
  transition: filter 0.4s ease;
}

img:hover {
  filter: grayscale(0%);
}
```





CSS Tooltips

Small info messages that appear on hover — built with **pure CSS, no JavaScript required**.

1

Position the Parent

Set `position: relative` on the wrapper element

2

Position the Tooltip

Use `position: absolute` on the tooltip text to place it precisely

3

Toggle Visibility

Use `visibility: hidden` by default, switch to `visible` on `:hover`

4

Smooth Fade

Add `opacity` + `transition: opacity 0.3s ease` for a polished reveal

Tooltip Arrow with Pseudo-Elements

Use `::after` on the tooltip element to create a CSS-only arrow — no extra HTML needed.

- Set `border-color: black transparent transparent transparent` to form a **downward triangle**
- Position with `top: 100%; left: 50%`
- Center it using `transform: translateX(-50%)`

> Pseudo-elements are a powerful CSS tool — combine them with `position` and `border` tricks to create shapes without images.

Arrow Code

```
.tooltip::after {
  content: "";
  position: absolute;
  top: 100%;
  left: 50%;
  transform: translateX(-50%);
  border-width: 6px;
  border-style: solid;
  border-color: black
  transparent
  transparent
  transparent;
}
```

Putting It All Together: Product Card

Gradient

Card or hero background

Transform + Transition

Card lifts smoothly on hover

Animation

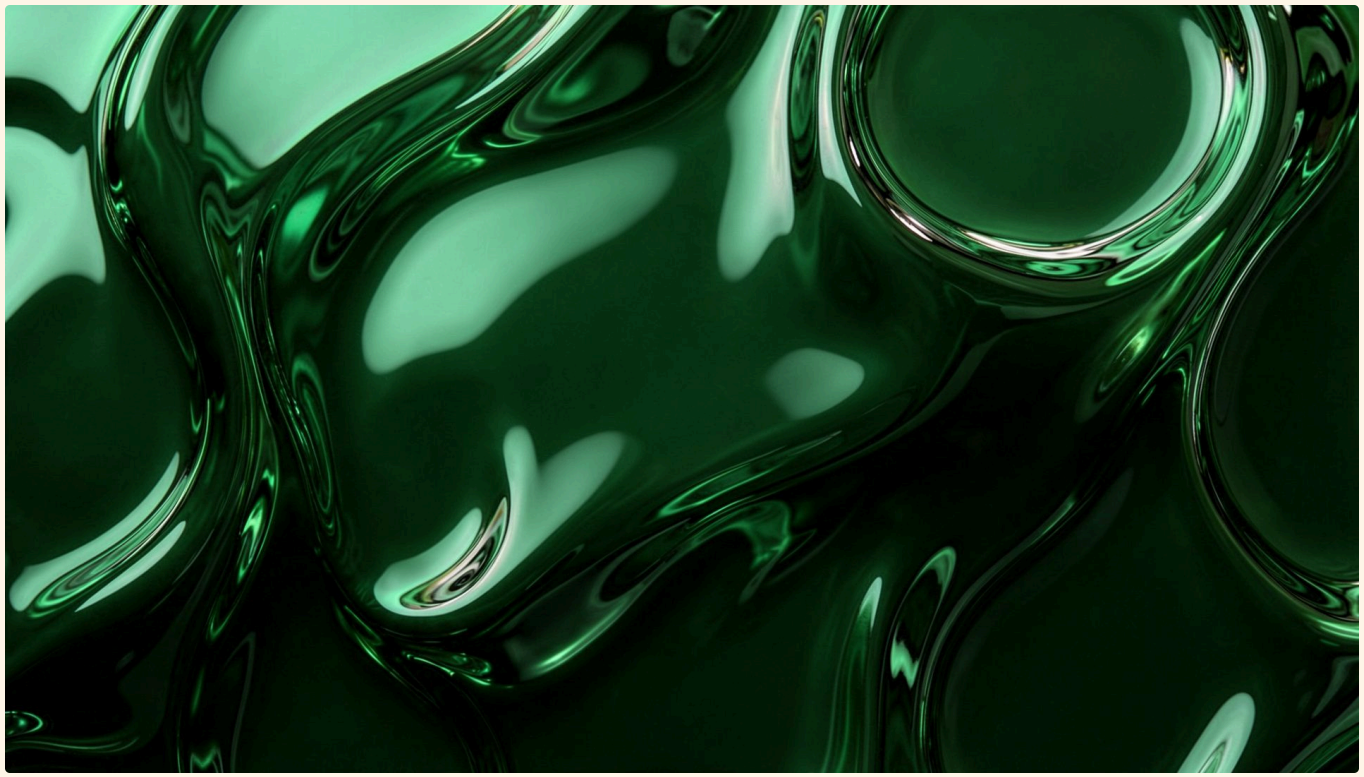
Discount badge pulses continuously

Filter

Product image shifts from grayscale to color on hover

Tooltip

Extra info appears over an icon



Mini Project: Modern Product Showcase

Build a full webpage using **HTML & CSS only**.

Required Page Sections

- Header & navigation
- Hero section
- Product cards
- Footer

CSS Requirements

- Gradients, transformations & transitions
- At least one `@keyframes` animation
- At least two filters
- At least two tooltips
- Hover effects throughout
- Responsive media queries

Learning Outcomes

Modern Backgrounds

Create linear, radial, and conic gradients with precise color stops

Element Transforms

Move, rotate, scale, and skew elements — individually and combined

Smooth UI Interactions

Build polished hover effects with transitions and transform pairings

Multi-Stage Animations

Design looping and directional animations using `@keyframes`

Visual Filters & Tooltips

Apply CSS filters, combine them with transitions, and build accessible tooltips with pseudo-elements